

DIGONTOM PRIVATE LIMITED

DEVELOPER PROJECT REPORT

Comprehensive Multi-Modal Transportation Booking Platform

Proposal ID	DPL/2026/TRANSPORT-001
Document Version	3.0
Date	April 30, 2026
Prepared By	Digontom Private Limited
Classification	Confidential
Validity	30 Days from Date of Issue

Bus | Car | Long-Journey Scheduling | Web & Mobile

1. PROJECT OVERVIEW

This document serves as the complete technical reference for the development team building the Digontom Multi-Modal Transportation Booking Platform. It details every module, data model, API contract, architecture decision, workflow, and deployment strategy required to deliver the platform.

1.1 Project Goals

- Build a unified transportation super-app for the Indian market covering Bus, Car, and Long-Journey scheduling.
- Deliver native-quality experiences on Android, iOS, and Web from a single React Native + Next.js codebase.
- Support 10,000+ concurrent users from day one with auto-scalable microservices.
- Integrate Indian payment rails: Razorpay UPI, Cards, Net Banking, and Wallets.
- Achieve production-grade security: AES-256, TLS 1.3, PCI-DSS, and number masking.

1.2 Core Platform Components

#	Component	Purpose	Platforms
1	Passenger App	Book buses, cars; plan long journeys	Android, iOS, Web
2	Driver / Operator App	Accept rides; manage bus schedules	Android, iOS
3	Admin Panel	Manage fleet, revenue, and analytics	Web Dashboard

2. TECHNOLOGY STACK

The platform uses a modern, scalable stack chosen for performance, developer productivity, and suitability for the Indian market.

Layer	Technology & Rationale
Mobile Apps (Android & iOS)	React Native — single codebase for both platforms; enables code sharing with web layer. Use Expo managed workflow for faster builds.
Web Frontend	React.js + Next.js 14 (App Router) + Tailwind CSS — SSR for SEO-friendly booking pages, ISR for route/schedule pages.
Backend API	Node.js with NestJS — modular microservices; each service is independently deployable. TypeScript throughout.
Primary Database	PostgreSQL 16 — relational data: bookings, users, payments, routes. Use connection pooling via PgBouncer.
Secondary Database	MongoDB 7 — logs, analytics events, reviews, geospatial ride data. Atlas Search for full-text queries.

Cache & Real-Time State	Redis 7 — session management, seat locks (5-min TTL), live driver locations (Geo commands), pub/sub.
Real-Time Communication	Socket.IO over WebSockets — live GPS tracking, ride status updates, driver-passenger messaging.
Maps & GPS	Google Maps Platform (primary): Maps JS API, Directions API, Geocoding API. Mapbox GL JS as fallback.
Payment Gateway	Razorpay — UPI, Cards, Net Banking, Wallets; PayU as secondary. Webhook-driven reconciliation.
SMS / OTP	MSG91 (primary, India) + Twilio (international fallback). 6-digit OTP with 5-min expiry.
Push Notifications	Firebase Cloud Messaging (Android & Web) + APNs (iOS) via unified NestJS notification service.
Cloud Hosting	Hostinger VPS — containerised services via Docker Compose; Nginx reverse proxy; Let's Encrypt SSL.
CI/CD Pipeline	GitHub Actions — build → lint → test → Docker build → deploy to VPS on merge to main.
Monitoring & Alerts	Grafana dashboards + Prometheus metrics + Sentry error tracking. Uptime Robot for public endpoint monitoring.

3. SYSTEM ARCHITECTURE

3.1 Microservices Breakdown

Each microservice owns its own database schema, exposes REST + WebSocket APIs, and communicates with peers via a message bus (Redis pub/sub for real-time, REST for synchronous calls).

Service	Port	Responsibilities
auth-service	3001	JWT issuance, OTP verification, OAuth2 (Google/Apple/Facebook), token refresh, session management
user-service	3002	User profiles, KYC uploads, saved addresses, emergency contacts, wallet balance
bus-service	3003	Routes, schedules, fleet management, seat layouts, boarding/dropping points, operator management
booking-service	3004	Seat reservation, booking lifecycle, e-ticket generation, QR codes, cancellation, refunds
ride-service	3005	Car ride requests, driver matching algorithm, surge pricing engine, ride lifecycle management
journey-service	3006	Multi-leg itinerary builder, Smart Schedule Optimizer, co-traveller management, journey alerts
payment-service	3007	Razorpay/PayU integration, wallet top-up/deduction, cashback engine, GST calculation, invoicing
tracking-service	3008	Real-time GPS ingestion from drivers (Socket.IO), location broadcasting to passengers, ETA calculation
notification-service	3009	FCM/APNs push, MSG91 SMS, email via SendGrid; template management, scheduled reminders
admin-service	3010	Admin REST API for all management operations, analytics queries, report generation
driver-service	3011	Driver onboarding, document KYC queue, earnings calculation, payout scheduling, incentive tracking
api-gateway	80/443	Nginx + Node.js BFF — auth middleware, rate limiting, request routing, response caching

3.2 Data Flow Overview

All client requests hit the API Gateway (Nginx). The gateway validates the JWT, applies rate limiting, and routes to the appropriate microservice. Real-time events (GPS, ride status) go directly via Socket.IO to the tracking-service. The booking and payment services coordinate via synchronous REST calls within the private VPC.

3.3 Redis Usage Patterns

Key Pattern	TTL	Purpose
seat:lock:{bookingId}:{seatId}	300s	Prevents double-booking during payment window
session:{userId}	86400s	JWT session; invalidated on logout
driver:loc:{driverId}	30s	Latest lat/lng from driver app GPS ping
surge:{zoneId}	300s	Surge multiplier per zone, recomputed every 5 min
otp:{phone}	300s	OTP value; max 3 attempts before block
rate:{ip}	60s	Request count for IP-based rate limiting (100 req/min)

4. DATABASE SCHEMA (PostgreSQL)

The following tables form the core relational schema. Each microservice accesses only its own schema; cross-service data is fetched via API, not direct DB joins.

4.1 Users & Auth

Column	Type	Notes
id	UUID PK	Generated via gen_random_uuid()
phone	VARCHAR(15) UNIQUE	E.164 format; primary login identifier
email	VARCHAR(255) UNIQUE	Nullable; used for email login / OTP
name	VARCHAR(255)	Display name
avatar_url	TEXT	S3/CDN URL
kyc_status	ENUM	pending submitted verified rejected
wallet_balance	NUMERIC(12,2)	In INR; never goes negative
preferred_language	VARCHAR(10)	BCP-47 locale code, default 'en'
created_at	TIMESTAMPTZ	Auto-set on insert
is_active	BOOLEAN	Soft-delete flag

4.2 Bus Routes & Schedules

Column	Type	Notes
--------	------	-------

id	UUID PK	
operator_id	UUID FK → operators	Bus operator company
route_name	VARCHAR(255)	e.g. 'Kolkata → Siliguri'
source_city	VARCHAR(100)	
destination_city	VARCHAR(100)	
departure_time	TIME	Scheduled departure
arrival_time	TIME	Scheduled arrival
bus_type	ENUM	AC_SLEEPER AC_SEATER NON_AC_SLEEPER NON_AC_SEATER
total_seats	INTEGER	Total capacity
base_fare	NUMERIC(10,2)	Per-seat base price in INR
amenities	JSONB	Array of strings: ['Charging Point', 'Blanket', 'Water']
cancellation_policy	JSONB	Tiered refund rules by hours before departure
is_active	BOOLEAN	

4.3 Bookings

Column	Type	Notes
id	UUID PK	
user_id	UUID FK → users	Passenger
booking_type	ENUM	BUS CAR JOURNEY_LEG
schedule_id	UUID FK	Points to bus schedule or car ride
journey_id	UUID FK	Nullable; parent multi-leg journey
seat_numbers	INTEGER[]	Array of booked seat numbers
boarding_point	VARCHAR(255)	Selected boarding stop
dropping_point	VARCHAR(255)	Selected dropping stop
passenger_details	JSONB	Name, age, gender per seat
fare_breakdown	JSONB	Base fare, GST, service fee, discount
total_amount	NUMERIC(10,2)	Total charged in INR
payment_id	UUID FK → payments	
status	ENUM	PENDING CONFIRMED CANCELLED COMPLETED

qr_code	TEXT	Base64 QR for conductor scan
travel_date	DATE	
created_at	TIMESTAMPTZ	

4.4 Car Rides

Column	Type	Notes
id	UUID PK	
passenger_id	UUID FK → users	
driver_id	UUID FK → drivers	Assigned after matching
ride_type	ENUM	MINI SEDAN SUV PREMIUM OUTSTATION HOURLY
pickup_location	POINT	PostGIS geometry
drop_location	POINT	PostGIS geometry
pickup_address	TEXT	Human-readable
drop_address	TEXT	
distance_km	NUMERIC(8,2)	Calculated after trip
fare_estimate	NUMERIC(10,2)	Shown before booking
fare_final	NUMERIC(10,2)	Actual charge post-trip
surge_multiplier	NUMERIC(4,2)	1.0 = no surge
status	ENUM	REQUESTED ACCEPTED ARRIVED IN_PROGRESS COMPLETED CANCELLED
scheduled_at	TIMESTAMPTZ	Nullable; for pre-booked rides
started_at / ended_at	TIMESTAMPTZ	Actual trip timestamps

5. API REFERENCE (Core Endpoints)

All APIs are RESTful, versioned at /api/v1/, and require Authorization: Bearer {JWT} unless marked [public]. Responses use standard JSON with { success, data, error } envelope.

5.1 Auth Service

Method	Endpoint	Description
--------	----------	-------------

POST	/auth/otp/send [public]	Send OTP to phone number. Body: { phone }. Rate-limited: 3 req/15min per phone.
POST	/auth/otp/verify [public]	Verify OTP, return access + refresh tokens. Body: { phone, otp }.
POST	/auth/refresh [public]	Rotate access token using refresh token. Body: { refreshToken }.
POST	/auth/logout	Invalidate session; delete Redis session key.
POST	/auth/social [public]	OAuth2 login. Body: { provider: 'google' 'apple' 'facebook', idToken }.

5.2 Bus Service

Method	Endpoint	Description
GET	/bus/search [public]	Query: source, destination, date, returnDate?. Returns available buses sorted by departure time.
GET	/bus/schedule/:id/seats	Real-time seat availability map for a schedule. Returns array of seat objects with status.
POST	/bus/book	Create booking. Body: { scheduleId, seats[], boardingPoint, droppingPoint, passengerDetails[], promoCode? }.
GET	/bus/booking/:id	Fetch booking details + e-ticket QR.
POST	/bus/booking/:id/cancel	Cancel booking; trigger refund per cancellation policy.
GET	/bus/track/:scheduleId	Live GPS position of bus on travel day. Polls tracking-service.

5.3 Car Ride Service

Method	Endpoint	Description
POST	/ride/estimate	Body: { pickup, drop, rideType }. Returns fare estimate + surge info.
POST	/ride/request	Create ride request. Driver matching triggered async. Returns { rideId, status: 'SEARCHING' }.
GET	/ride/:id/status	Poll ride status and driver details. Switch to WebSocket after driver assigned.
POST	/ride/:id/cancel	Cancel ride; apply cancellation fee if driver already assigned.
POST	/ride/:id/rate	Submit post-trip rating. Body: { rating: 1-5, comment? }.
POST	/ride/schedule	Book a ride up to 7 days in advance. Body: { ...rideRequest, scheduledAt }.

5.4 Long-Journey Service

Method	Endpoint	Description
POST	/journey/plan	Body: { origin, destination, travelDate, flexibility: ±days }. Returns multi-leg itinerary suggestions.
POST	/journey/optimize	Body: { legs[] }. AI optimizer adjusts departure times to minimize layovers.
POST	/journey/book	Atomic multi-leg booking — wraps bus and ride booking in a single transaction.
GET	/journey/:id	Full itinerary with real-time status per leg.
GET	/journey/:id/share	Generate shareable link for family tracking.
PUT	/journey/:id/travellers	Add/remove co-travellers. Body: { travellers: [{name, phone}] }.

6. WEBSOCKET EVENTS (Socket.IO)

The tracking-service runs Socket.IO. Clients connect with their JWT in the auth handshake query param. Namespaces separate concerns.

6.1 Passenger Namespace: /passenger

Event Name	Direction	Payload / Purpose
join:ride	Client → Server	{ rideId } — Join room for this ride's updates
ride:driver_assigned	Server → Client	{ driverId, name, vehicle, photo, eta, phone_masked } — Driver matched
ride:location_update	Server → Client	{ lat, lng, bearing, eta } — Every 5s while ride in progress
ride:status_change	Server → Client	{ status } — ARRIVED IN_PROGRESS COMPLETED
bus:location_update	Server → Client	{ lat, lng, eta, delay_minutes } — Live bus tracking on travel day
journey:leg_alert	Server → Client	{ legId, type, message } — Delay / gate change / departure reminder
ride:chat	Bidirectional	{ message, timestamp } — In-app driver-passenger messaging

6.2 Driver Namespace: /driver

Event Name	Direction	Payload / Purpose
driver:go_online	Client → Server	{ driverId, lat, lng } — Mark available; enter matching pool
driver:location	Client → Server	{ lat, lng, bearing, speed } — GPS ping every 5s while online
ride:new_request	Server → Client	{ rideId, pickup, drop, fare, distance } — 30-sec accept window
ride:accepted	Client → Server	{ rideId } — Driver accepts; match confirmed
ride:rejected	Client → Server	{ rideId } — Driver rejects; re-queue for next driver
ride:start	Client → Server	{ rideId } — Passenger boarded; trip begins
ride:end	Client → Server	{ rideId, finalLat, finalLng } — Trip complete; fare calculated

7. KEY ALGORITHMS & BUSINESS LOGIC

7.1 Driver Matching Algorithm

When a ride request is created, the ride-service runs the following matching logic:

1. Query Redis GEORADIUS for online drivers within 5km of pickup, sorted by distance.
2. Filter by vehicle type matching the requested ride_type.
3. For each candidate driver, check acceptance rate (reject if < 60% in last 30 days).
4. Send ride:new_request WebSocket event to the top candidate with a 30-second timer.
5. If rejected or timeout, move to next candidate. After 3 failures, expand radius to 10km.
6. On acceptance, lock the match, update ride status to ACCEPTED, and notify passenger.

7.2 Seat Lock Mechanism

To prevent double-booking during the payment window:

7. When a user selects seats, call POST /bus/seats/lock with { scheduleId, seatIds[], userId }.
8. booking-service calls Redis SETNX seat:lock:{scheduleId}:{seatId} {userId} EX 300.
9. If any seat lock fails (already locked), return 409 Conflict with { failedSeats[] }.
10. On payment success, convert locks to confirmed booking records.
11. On payment failure or timeout, locks auto-expire; seats return to available pool.

7.3 Surge Pricing Engine

Surge multiplier is computed every 5 minutes per geofenced zone:

12. Count active_ride_requests and online_drivers within each zone polygon (PostGIS).
13. Compute demand_ratio = active_requests / online_drivers.
14. Apply multiplier: 1.0 (ratio < 1.5), 1.2 (1.5–2.0), 1.5 (2.0–3.0), 2.0 (> 3.0).
15. Store result in Redis surge:{zoneId} with 300s TTL.
16. Display multiplier with clear notification before passenger confirms booking.

7.4 Long-Journey Smart Optimizer

Given a multi-leg itinerary, the optimizer:

17. Fetches all candidate schedules for each leg from bus-service.
18. Builds a directed graph where nodes = stop + time and edges = available schedules.
19. Runs Dijkstra's algorithm weighted by total_layover_time + price.
20. Flags any connection with layover < 30 minutes as 'risky' and suggests safer alternatives.
21. Returns top 3 itinerary combinations sorted by (price × 0.4 + layover_time × 0.6).

8. SECURITY IMPLEMENTATION

Area	Implementation Details
Data Encryption	AES-256-GCM at rest for sensitive fields (Aadhaar, account numbers). TLS 1.3 in transit; HSTS header enforced.
Authentication	JWT (RS256 asymmetric keys, 15-min access token, 7-day refresh token). OTP via MSG91 with 5-min expiry and 3-attempt limit.
Payment Security	PCI-DSS compliant via Razorpay. No card data stored on Digontom servers. Webhook signatures verified with HMAC-SHA256.
API Security	Rate limiting: 100 req/min per IP (Nginx), 30 req/min per user (NestJS throttler). Input validation via class-validator. CORS whitelist. API key rotation every 90 days.
Number Masking	Driver-passenger calls routed via MSG91 virtual numbers. Real phone numbers never exposed to either party.
SQL Injection	All DB queries use parameterized statements via TypeORM. No raw query string interpolation.
Data Backup	Daily pg_dump snapshots to S3-compatible storage (Hostinger Object Storage). 30-day retention. Automated restore drills monthly.
Privacy	India IT Act 2000 compliant. GDPR-ready: right to erasure (soft delete + PII anonymisation after 90 days). Privacy policy displayed at signup.

9. SPRINT PLAN & MILESTONES

Total project duration: 75 days. 9 team members. 2-day sprints. Daily standups. Weekly client demos.

Phase	Duration	Deliverables & Developer Tasks
Phase 1 Discovery	Days 1–5	Finalise requirements; DB schema design; Figma wireframes; NestJS project scaffold; GitHub repo + branch strategy; Docker Compose local dev setup.
Phase 2 UI/UX Design	Days 6–12	Complete Figma designs for all 50+ screens across Passenger App, Driver App, and Admin Panel. Design system: colours, typography, components. Dev handoff tokens.
Phase 3 Backend Dev	Days 13–30	auth-service, user-service, bus-service, booking-service, payment-service APIs. PostgreSQL schema migrations. Razorpay webhook integration. Redis seat locking.
Phase 4 Passenger App	Days 20–40	React Native: Home, Auth, Bus Search, Seat Map, Booking, Wallet, Profile, Trip History screens. Google Maps integration. Socket.IO live tracking.
Phase 5 Driver App	Days 31–45	React Native Driver App: Onboarding/KYC, Go Online, Ride Request modal, In-App Navigation, Earnings Dashboard, Bus Operator schedule manager.
Phase 6 Admin Panel	Days 36–50	Next.js Admin Dashboard: Overview, Real-Time Map, Bus/Route Management, Driver Approval Queue, Revenue Charts, User Management, Notification Broadcasts.
Phase 7 Journey Module	Days 46–55	journey-service: multi-leg planner, Dijkstra optimizer, co-traveller tracking, Journey Calendar UI, offline itinerary download, WhatsApp share integration.
Phase 8 QA & Testing	Days 56–68	Manual testing all user flows. Jest unit tests (≥80% coverage). Playwright E2E tests. k6 load testing: 10,000 concurrent users. OWASP security audit. Bug fixes.
Phase 9 Deployment	Days 69–75	Docker build + push. Nginx config. SSL via Let's Encrypt. VPS deployment. Play Store + App Store submission. Soft launch with 100 beta users. Monitoring setup.

10. TEAM COMPOSITION & RESPONSIBILITIES

#	Role	Detailed Responsibilities
1	Project Manager (1)	Sprint planning in Jira, daily standups, client weekly demos, risk management, timeline tracking, blocker resolution.
2	UI/UX Designer (1)	Figma design for all screens, design system tokens, developer handoff, responsive layouts, usability review on each sprint.
3	Backend Developer A (1 of 2)	auth-service, user-service, payment-service, notification-service. JWT/OTP flows, Razorpay webhooks, wallet engine.
4	Backend Developer B (1 of 2)	bus-service, booking-service, ride-service, journey-service, tracking-service. Matching algo, seat lock, GPS ingestion, optimizer.
5	Mobile Developer A (1 of 2)	React Native Passenger App — all screens, Google Maps SDK, Socket.IO client, push notifications, offline mode.
6	Mobile Developer B (1 of 2)	React Native Driver App — KYC flow, ride request UI, Mapbox navigation, earnings dashboard, bus operator module.
7	Frontend Web Developer (1)	Next.js Passenger Web App (SSR/ISR booking pages) + Admin Panel (Next.js + Recharts + TanStack Table).
8	QA Engineer (1)	Test case authoring, manual regression, Jest + Playwright automation, k6 load scripts, security checklist, bug reporting.
9	DevOps Engineer (1)	Docker Compose + production Dockerfiles, GitHub Actions CI/CD, Nginx config, Hostinger VPS setup, Prometheus + Grafana, Sentry.

11. TESTING STRATEGY

11.1 Unit Testing (Jest + Supertest)

- Each service must achieve $\geq 80\%$ line coverage before merging to main.
- Mock Redis and PostgreSQL with in-memory equivalents (ioredis-mock, pg-mem).
- Test all business logic: surge calc, seat lock, fare breakdown, OTP expiry.

11.2 Integration Testing

- Spin up real PostgreSQL + Redis in GitHub Actions via `docker-compose.test.yml`.
- Test full booking flow end-to-end via Supertest against running NestJS app.
- Verify Razorpay webhook signature validation with test payloads.

11.3 E2E Testing (Playwright)

- Bus booking happy path: search $\rightarrow$ seat select $\rightarrow$ payment $\rightarrow$ ticket download.
- Car ride flow: request $\rightarrow$ driver match $\rightarrow$ trip start $\rightarrow$ end $\rightarrow$ rating.
- Admin panel: operator onboard $\rightarrow$ schedule create $\rightarrow$ booking list verification.

11.4 Load Testing (k6)

- Target: 10,000 virtual users, 30-minute sustained load test.
- Scenarios: concurrent bus searches, simultaneous seat lock requests, GPS ping flood.
- Acceptance criteria: p95 response time $< 500\text{ms}$, error rate $< 0.1\%$.

11.5 Security Testing

- OWASP Top 10 checklist per service: injection, broken auth, XSS, CORS, rate limiting.
- Burp Suite scan on all public endpoints.
- Verify seat locks cannot be bypassed by concurrent requests (race condition test).

12. DEPLOYMENT & DEVOPS

12.1 Environment Strategy

Environment	Branch	Purpose
Development	feature/* → dev	Local Docker Compose; each developer runs full stack locally.
Staging	dev → staging	Auto-deployed to staging VPS on every dev merge. QA runs here.
Production	staging → main	Manual promote after QA sign-off. Zero-downtime rolling restart.

12.2 GitHub Actions CI Pipeline

22. Trigger: push to any branch or PR to dev/main.
23. Step 1 — Lint: ESLint + Prettier check (fail fast on violations).
24. Step 2 — Test: Jest unit tests + integration tests against Docker services.
25. Step 3 — Build: Docker image built and tagged with git SHA.
26. Step 4 — Push: Image pushed to GitHub Container Registry (ghcr.io).
27. Step 5 — Deploy: SSH to VPS, pull latest image, docker-compose up -d --no-deps {service}.

12.3 Nginx Configuration

- Reverse proxy: route /api/v1/auth → auth-service:3001, /api/v1/bus → bus-service:3003, etc.
- WebSocket upgrade: proxy_http_version 1.1 + Upgrade/Connection headers for Socket.IO.
- SSL: Let's Encrypt via Certbot; auto-renew cron. HSTS header: max-age=31536000.
- Rate limiting: limit_req_zone by \$binary_remote_addr at 100 req/min for /api/ routes.
- Gzip compression for API responses > 1KB. Cache static Next.js assets for 1 year.

12.4 Monitoring Stack

- Prometheus scrapes /metrics from each NestJS service (prom-client middleware).
- Grafana dashboards: requests/sec, p95 latency, error rate, DB connection pool, Redis memory.
- Sentry: error tracking with source maps for both backend and React Native apps.
- Uptime Robot: pings /health endpoint every 60s; PagerDuty alert if 2 consecutive failures.

13. THIRD-PARTY INTEGRATIONS

Service	Used For	Integration Notes
Razorpay	Payments	Use Orders API → Payment Links for bus; Payment Intent for rides. Webhook: payment.captured, payment.failed. Store razorpay_order_id and razorpay_payment_id in payments table.
Google Maps	Maps & Routing	Maps JS API (web), React Native Maps (mobile). Directions API for route polylines. Geocoding API for address resolution. Distance Matrix for fare estimates.
Mapbox	Fallback Maps	Used in Driver App for turn-by-turn navigation. Mapbox Navigation SDK (React Native). Switch logic: if Google Maps SDK fails, fallback to Mapbox.
MSG91	SMS/OTP	Transactional OTP flow: send → verify API. Use flow ID for OTP templates. Fallback to Twilio if MSG91 API returns 5xx.
Firebase FCM	Push Notifs (Android/Web)	Use firebase-admin SDK on notification-service. Send to device token or topic. Handle token refresh: update token on app open.
APNs	Push Notifs (iOS)	Use @parse/node-apn library. Certificates rotated annually. Background push for silent location updates.
SendGrid	Email	Transactional emails: booking confirmation, e-ticket PDF, OTP fallback, cancellation notice. Use dynamic templates.
Sentry	Error Monitoring	NestJS: @sentry/node. React Native: @sentry/react-native with source maps upload in CI. Alert on new error or spike.

15. DELIVERABLES CHECKLIST

#	Deliverable	Status
1	Passenger App — Android + iOS (React Native)	✓ Included
2	Passenger Web App (Next.js + React)	✓ Included
3	Driver / Operator App — Android + iOS	✓ Included
4	Admin Panel (Next.js Web Dashboard)	✓ Included
5	Bus Booking Module with interactive seat map	✓ Included
6	Long-Journey Scheduling Module	✓ Included
7	Car Ride Booking Module (Uber-style)	✓ Included
8	Razorpay Payment Gateway (UPI, Cards, Wallets)	✓ Included
9	Google Maps + Mapbox Integration	✓ Included
10	In-App Wallet & Cashback System	✓ Included
11	Push Notifications (FCM + APNs)	✓ Included
12	Real-Time GPS Tracking (Socket.IO)	✓ Included
13	Analytics & Revenue Dashboard	✓ Included
14	UI/UX Design in Figma (all screens)	✓ Included
15	QA, Load Testing & Security Audit	✓ Included
16	Deployment to Play Store, App Store & Web	✓ Included
17	Complete Source Code Handover	✓ Included
18	3 Months Free Bug Support (Post-Launch)	✓ Included
19	Technical Documentation & API Docs	✓ Included